Code: https://github.com/zhurleonid/corporate-risk-management
(`homework_6/hw6_code.ipynb`)

The write-up is below and the code is appended at the end. The derivations (VaR of the linearized loss, the conditional shock, the EWMA update after the shock) are handwritten and attached in front.

Stress testing a market-cap-weighted MSFT / AAPL portfolio

Set-up

$V_t = \$1{,}000{,}000$, weighted by the 09/01/2016 market caps ($\$448.77$bn MSFT, $\$575.11$ bn AAPL). Index (1) is MSFT, (2) is AAPL.

$$w = (0.4383,\ 0.5617), \qquad \theta_t = S_t\lambda_t = V_t\, w = (438{,}303.32,\ 561{,}696.68).$$

$\alpha = 0.95$, $K = 10$, $\lambda = \theta = 0.97$, $\hat{M} = 50{,}000$.

Data. Daily adjusted closes from Yahoo Finance, 09/01/2011 to 09/01/2016 (`stocks_daily.csv`): 1259 prices, 1258 log returns per stock. Adjusted prices are needed because of AAPL's 7-for-1 split in June 2014.

Step 1. EWMA mean and covariance

$\mu_{s+\Delta} = \lambda\mu_s + (1-\lambda)X_s, \qquad \Sigma_{s+\Delta} = \theta\Sigma_s + (1-\theta)(X_s-\mu_s)(X_s-\mu_s)^\top, \qquad \mu_0 =$

Pseudocode

```
mu = (0, 0),  Sigma = 2x2 zero matrix
for s = 1 ... 1258:
    d     = x_s - mu                        deviation from the
OLD mean
    Sigma = theta*Sigma + (1 - theta) * d d^T
    mu    = lambda*mu + (1 - lambda) * x_s
output mu_{t+D}, Sigma_{t+D}                t = 09/01/2016
```

Results

$$\mu_{t+\Delta} = \begin{pmatrix} 1.4045 \times 10^{-3} \\ 1.1655 \times 10^{-3} \end{pmatrix}, \qquad \Sigma_{t+\Delta} = \begin{pmatrix} 1.2627 \times 10^{-4} & 3.3241 \times 10^{-5} \\ 3.3241 \times 10^{-5} & 1.5204 \times 10^{-4} \end{pmatrix}$$

$\sigma_{t+\Delta} = (0.011237,\ 0.012331)$ daily (17.84%, 19.57% annualized), $\rho_{t+\Delta} = 0.2399$.

Step 2. VaR before the shock

> **By hand, attached:** $\mathrm{VaR}_\alpha(L^{\mathrm{lin}}_{t+\Delta}) = -\theta_t^\top \mu_{t+\Delta} + \sqrt{\theta_t^\top \Sigma_{t+\Delta}\theta_t}\;\Phi^{-1}(\alpha)$.

$\mathrm{VaR}_{0.95}(L^{\mathrm{lin}}_{t+\Delta})$	$14{,}211.95$
$\mathrm{VaR}_{0.95}(L^{\mathrm{lin}}_{t+10\Delta}) = \sqrt{10}\,\mathrm{VaR}_{0.95}(L^{\mathrm{lin}}_{t+\Delta})$	$44{,}942.15$
Regulatory capital $3 \times \mathrm{VaR}_{0.95}(L^{\mathrm{lin}}_{t+10\Delta})$	$134{,}826.44$

In dollars.

Step 3. The shock

> **By hand, attached:** $x_{(1)} = \mu_{t+\Delta,(1)} - 5\sigma_{t+\Delta,(1)}\big(\rho_{t+\Delta} + \sqrt{1-\rho^2_{t+\Delta}}\big)$ and the EWMA update.

(i) $x_{(2)} = \mu_{t+\Delta,(2)} - 5\sigma_{t+\Delta,(2)}$	-0.060488
(ii) $\tilde\mu_{t+\Delta,(1)}$	-0.012074
$\tilde\sigma_{t+\Delta,(1)}$	0.010909
$x_{(1)} = \tilde\mu_{t+\Delta,(1)} - 5\tilde\sigma_{t+\Delta,(1)}$	-0.066619

The loss on the shock day is $-\theta_t^\top x = 63{,}175.01$.

(iii) EWMA update with $x = (x_{(1)}, x_{(2)})$:

$$\mu_{t+2\Delta} = \begin{pmatrix} -6.362 \times 10^{-4} \\ -6.841 \times 10^{-4} \end{pmatrix}, \qquad \sigma_{t+2\Delta} = \begin{pmatrix} 0.016165 \\ 0.016171 \end{pmatrix}$$

$$\rho_{t+2\Delta} = 0.6046.$$

The shock raises both volatilities (to about 25.7% annualized) and the correlation from 0.24 to 0.60.

Steps 4-5. Simulation

Pseudocode

```
inputs: mu_1 = mu_{t+D}, Sigma_1 = Sigma_{t+D} (step 1)
        shock x (step 3),  theta_t = V * w

for m = 1 ... M:          (all paths at once in the code)
    for k = 1 ... K:
        if k == 1:
```

```
            X_k = x
        else:
            X_k = mu_k + chol(Sigma_k) Z,   Z ~ N(0, I)
        if k < K:
            d           = X_k - mu_k
            Sigma_{k+1} = theta*Sigma_k + (1 - theta) * d d^T
            mu_{k+1}    = lambda*mu_k + (1 - lambda) * X_k
    l_m = -theta_t^T (X_1 + ... + X_K)

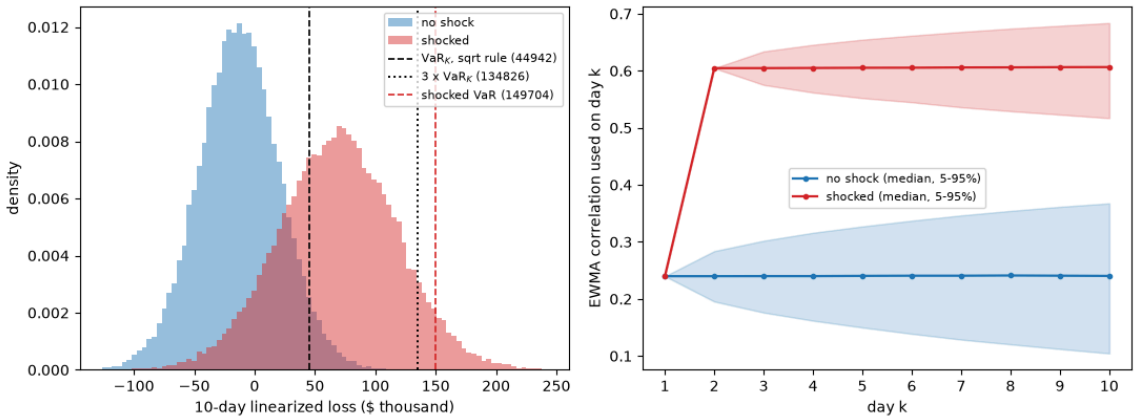
(a) mean of l_1 ... l_M
(b) VaR = sorted l at position ceil(M*alpha) = 47,500
(c) 100/M * #{m : l_m > sqrt(10) * VaR_1}
(d) 100/M * #{m : l_m > 3 * sqrt(10) * VaR_1}
```

Results

(a) average 10-day loss	69,178.06
(b) 10-day $\mathrm{VaR}_{0.95}$	149,703.73
(c) losses $>$ 44,942.15 (initial 10-day VaR)	69.19%
(d) losses $>$ 134,826.44 (regulatory capital)	8.77%

$\hat{M} = 50{,}000$ *paths, in dollars. (c) is 34,593 paths, (d) is 4,386 paths.*

The shocked 10-day VaR is 3.33 times the initial 10-day VaR and exceeds the regulatory capital. The shock day alone loses $63{,}175$, more than the initial 10-day VaR, so most paths exceed it (69%). The regulatory capital is exceeded on 8.8% of paths, compared with 0.006% in the same simulation without the shock. After the shock the EWMA correlation stays near 0.60 for the rest of the horizon (right panel), so the two stocks no longer diversify each other.



Left: 10-day linearized loss with and without the shock, with the initial $\sqrt{10}$ VaR, the regulatory capital and the shocked VaR. Right: EWMA correlation used on each day, median and 5-95% band across paths.

Appendix: source code

The code below produces every figure and number above.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm

ALPHA = 0.95
K = 10              # horizon in days
LAM = 0.97          # EWMA mean
THETA = 0.97        # EWMA covariance (not the dollar position theta_t)
V0 = 1_000_000
M_PATHS = 50_000
N_SIG = 5           # size of the shock, in standard deviations

plt.rcParams['figure.figsize'] = (11, 5)


def var_emp(losses, a):
    # VaR from a sample: the ceil(n*a)-th order statistic
    ls = np.sort(np.asarray(losses))
    n = len(ls)
    return ls[int(np.ceil(n * a)) - 1]
```

In [2]:
```
# Daily closes (split and dividend adjusted), saved to csv so this runs
# offline. To pull fresh data:
#     import yfinance as yf
#     yf.download(['MSFT', 'AAPL'], start='2011-09-01', end='2016-09-02',
#                 auto_adjust=True)['Close']
# (Yahoo's end date is exclusive, hence 09/02 to include 09/01.)
# AAPL split 7-for-1 in June 2014: unadjusted closes would show a fake
# -195% log return.
prices = pd.read_csv('stocks_daily.csv', parse_dates=['Date']).set_index('Da
tickers = ['MSFT', 'AAPL']
prices = prices[tickers]

print('first five observations'); print(prices.head())
print('\nlast five observations'); print(prices.tail())
print('\nN =', len(prices), 'trading days,',
      prices.index[0].date(), '->', prices.index[-1].date())
```

```
first five observations
                MSFT       AAPL
Date
2011-09-01  20.245007  11.396465
2011-09-02  19.928308  11.187697
2011-09-06  19.704319  11.357882
2011-09-07  20.082808  11.483206
2011-09-08  20.252731  11.489485

last five observations
                MSFT       AAPL
Date
2016-08-26  51.528732  24.455643
2016-08-29  51.590897  24.428202
2016-08-30  51.404434  24.240677
2016-08-31  51.022594  24.263550
2016-09-01  51.138039  24.407616

N = 1259 trading days, 2011-09-01 -> 2016-09-01
```

In [3]:
```
# log returns X_s = ln(S_s / S_{s-1}), one row per day
X = np.log(prices / prices.shift(1)).dropna()
x_hist = X.values
n_hist = len(x_hist)
print('log returns:', n_hist, 'rows,', X.index[0].date(), '->', X.index[-1].
print(X.describe().loc[['mean', 'std', 'min', 'max']].T)
```

```
log returns: 1258 rows, 2011-09-02 -> 2016-09-01
          mean       std       min       max
MSFT  0.000737  0.014914 -0.121033  0.099413
AAPL  0.000605  0.016777 -0.131885  0.085022
```

In [4]:
```
mcap = pd.Series({'MSFT': 448.77, 'AAPL': 575.11})      # $ billion, 09/01/2
w = (mcap / mcap.sum())[tickers]
pos = V0 * w.values         # theta_t: dollars in each stock, held as fixed

print('weights w_i'); print(w.round(6))
print('\ndollar positions theta_t:', pos.round(2))
print('check: total = %.2f' % pos.sum())
```

```
weights w_i
MSFT    0.438303
AAPL    0.561697
dtype: float64

dollar positions theta_t: [438303.32 561696.68]
check: total = 1000000.00
```

In [5]:
```
mu = np.zeros(2)
Sig = np.zeros((2, 2))
for x in x_hist:
    d = x - mu                                          # uses mu_s, before it is
    Sig = THETA * Sig + (1 - THETA) * np.outer(d, d)
    mu = LAM * mu + (1 - LAM) * x

sd = np.sqrt(np.diag(Sig))
rho = Sig[0, 1] / (sd[0] * sd[1])
```

```
print('EWMA estimates as of 09/01/2016 (for the return over 09/01 -> 09/02)'
print('mu    =', mu)
print('Sigma =\n', Sig)
print('daily sd       =', sd.round(6))
print('annualized vol =', (sd * np.sqrt(252)).round(4))
print('correlation    = %.4f' % rho)
print('\nweight left on (mu_0, Sigma_0) = 0 after %d steps: %.1e'
      % (n_hist, LAM**n_hist))
```

```
EWMA estimates as of 09/01/2016 (for the return over 09/01 -> 09/02)
mu    = [0.00140454 0.00116553]
Sigma =
 [[1.26271806e-04 3.32407835e-05]
 [3.32407835e-05 1.52044263e-04]]
daily sd       = [0.011237 0.012331]
annualized vol = [0.1784 0.1957]
correlation    = 0.2399

weight left on (mu_0, Sigma_0) = 0 after 1258 steps: 2.3e-17
```

In [6]:
```
# L_lin = -theta' X  ~  N(-theta' mu, theta' Sigma theta)
z = norm.ppf(ALPHA)
loss_mean_1 = -pos @ mu
loss_sd_1 = np.sqrt(pos @ Sig @ pos)

VaR_1 = loss_mean_1 + loss_sd_1 * z
VaR_K = np.sqrt(K) * VaR_1              # square-root-of-time rule
reg_cap = 3 * VaR_K                     # regulatory capital, 3 x 10-day VaR

print('one-day loss: mean %.2f   sd %.2f' % (loss_mean_1, loss_sd_1))
print('VaR_0.95, 1-day            = %10.2f' % VaR_1)
print('VaR_0.95, %d-day (sqrt K)  = %10.2f' % (K, VaR_K))
print('regulatory capital 3 x VaR = %10.2f' % reg_cap)
```

```
one-day loss: mean -1270.29   sd 9412.54
VaR_0.95, 1-day            =   14211.95
VaR_0.95, 10-day (sqrt K)  =   44942.15
regulatory capital 3 x VaR =  134826.44
```

In [7]:
```
# (i) AAPL: 5 sigma down
x2 = mu[1] - N_SIG * sd[1]

# (ii) MSFT given X_(2) = x2: conditional normal, then a 5 sigma conditional
mu1_c = mu[0] + rho * sd[0] / sd[1] * (x2 - mu[1])
sd1_c = sd[0] * np.sqrt(1 - rho**2)
x1 = mu1_c - N_SIG * sd1_c

x_shock = np.array([x1, x2])
shock_loss = -pos @ x_shock

print('AAPL shock  x2 = %.6f   (%.2f%% simple return)' % (x2, 100 * np.expm1
print('MSFT | AAPL: conditional mean %.6f, conditional sd %.6f' % (mu1_c, sc
print('MSFT shock  x1 = %.6f   (%.2f%% simple return)' % (x1, 100 * np.expm1
print('  = %.2f unconditional sd below mu' % ((mu[0] - x1) / sd[0]))
print('linearized loss on the shock day = %.2f' % shock_loss)
```

```
print('  = %.2f x the one-day VaR, %.2f x the sqrt-rule 10-day VaR'
      % (shock_loss / VaR_1, shock_loss / VaR_K))
```

```
AAPL shock  x2 = -0.060488   (-5.87% simple return)
MSFT | AAPL: conditional mean -0.012074, conditional sd 0.010909
MSFT shock  x1 = -0.066619   (-6.44% simple return)
  = 6.05 unconditional sd below mu
linearized loss on the shock day = 63175.01
  = 4.45 x the one-day VaR, 1.41 x the sqrt-rule 10-day VaR
```

In [8]:
```
# (iii) EWMA update with the shocked returns -> mu_{t+2D}, Sigma_{t+2D}
d = x_shock - mu
Sig_2 = THETA * Sig + (1 - THETA) * np.outer(d, d)
mu_2 = LAM * mu + (1 - LAM) * x_shock
sd_2 = np.sqrt(np.diag(Sig_2))
rho_2 = Sig_2[0, 1] / (sd_2[0] * sd_2[1])

ann = np.sqrt(252)
print('                     before shock   after shock')
print('mu MSFT             %12.6f  %12.6f' % (mu[0], mu_2[0]))
print('mu AAPL             %12.6f  %12.6f' % (mu[1], mu_2[1]))
print('ann. vol MSFT       %12.4f  %12.4f' % (sd[0] * ann, sd_2[0] * ann))
print('ann. vol AAPL       %12.4f  %12.4f' % (sd[1] * ann, sd_2[1] * ann))
print('correlation         %12.4f  %12.4f' % (rho, rho_2))
print('one-day loss sd     %12.2f  %12.2f' % (loss_sd_1, np.sqrt(pos @ Sig_2
print('one-day loss mean   %12.2f  %12.2f' % (loss_mean_1, -pos @ mu_2))
```

```
                     before shock   after shock
mu MSFT                  0.001405     -0.000636
mu AAPL                  0.001166     -0.000684
ann. vol MSFT              0.1784        0.2566
ann. vol AAPL              0.1957        0.2567
correlation                0.2399        0.6046
one-day loss sd          9412.54      14509.78
one-day loss mean       -1270.29        663.07
```

In [9]:
```
def simulate(shock, rng, M=M_PATHS):
    mus = np.tile(mu, (M, 1))
    Sigs = np.tile(Sig, (M, 1, 1))
    X_sum = np.zeros((M, 2))
    corr_path = np.empty((M, K))           # correlation used on day k (k =
    for k in range(1, K + 1):
        corr_path[:, k - 1] = Sigs[:, 0, 1] / np.sqrt(Sigs[:, 0, 0] * Sigs[:
        if k == 1 and shock:
            Xk = np.tile(x_shock, (M, 1))
        else:
            # X = mu + C Z with C the Cholesky factor of each path's Sigma
            C = np.linalg.cholesky(Sigs)
            Z = rng.standard_normal((M, 2))
            Xk = mus + np.einsum('mij,mj->mi', C, Z)
        X_sum += Xk
        if k < K:                          # no update needed after day K
            dev = Xk - mus
            Sigs = THETA * Sigs + (1 - THETA) * np.einsum('mi,mj->mij', dev,
            mus = LAM * mus + (1 - LAM) * Xk
    # L_lin = -theta_t' sum_k X_{t+kD}, shares held fixed
    losses = -X_sum @ pos
```

```
    return losses, corr_path


rng = np.random.default_rng(731)
l_shock, corr_shock = simulate(True, rng)
l_base, corr_base = simulate(False, rng)
print('simulated %d paths of %d days, shocked and unshocked' % (M_PATHS, K))
```

```
simulated 50000 paths of 10 days, shocked and unshocked
```

In [10]:
```
mean_shock = l_shock.mean()
VaR_shock = var_emp(l_shock, ALPHA)
n_var = np.sum(l_shock > VaR_K)
n_reg = np.sum(l_shock > reg_cap)
freq_var = 100 * n_var / M_PATHS
freq_reg = 100 * n_reg / M_PATHS

print('(a) average %d-day loss = %.2f' % (K, mean_shock))
print('(b) %d-day VaR_0.95 = %.2f   (order statistic %d of %d)'
      % (K, VaR_shock, int(np.ceil(M_PATHS * ALPHA)), M_PATHS))
print('(c) losses above the sqrt-rule VaR (%.2f): %d paths = %.3f%%'
      % (VaR_K, n_var, freq_var))
print('(d) losses above the regulatory capital (%.2f): %d paths = %.3f%%'
      % (reg_cap, n_reg, freq_reg))

print('\nshocked loss sd %.2f' % l_shock.std(ddof=1))
print('shocked VaR / pre-shock VaR_K = %.3f' % (VaR_shock / VaR_K))
```

```
(a) average 10-day loss = 69178.06
(b) 10-day VaR_0.95 = 149703.73   (order statistic 47500 of 50000)
(c) losses above the sqrt-rule VaR (44942.15): 34593 paths = 69.186%
(d) losses above the regulatory capital (134826.44): 4386 paths = 8.772%

shocked loss sd 49081.95
shocked VaR / pre-shock VaR_K = 3.331
```

In [11]:
```
print('unshocked run: mean %.2f   sd %.2f   VaR %.2f'
      % (l_base.mean(), l_base.std(ddof=1), var_emp(l_base, ALPHA)))
print('               %% > VaR_K %.3f%%   %% > reg %.3f%%'
      % (100 * np.mean(l_base > VaR_K), 100 * np.mean(l_base > reg_cap)))
```

```
unshocked run: mean -12676.90   sd 33732.15   VaR 42603.88
               % > VaR_K 4.378%   % > reg 0.006%
```

In [12]:
```
fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))

kd = 1000                                       # plot in $ thousand
lo_edge = np.quantile(l_base, 0.0005)
hi_edge = np.quantile(l_shock, 0.9995)
bins = np.linspace(lo_edge, hi_edge, 90) / kd
ax[0].hist(l_base / kd, bins=bins, density=True, alpha=0.45, color='C0',
           label='no shock')
ax[0].hist(l_shock / kd, bins=bins, density=True, alpha=0.45, color='C3',
           label='shocked')
ax[0].axvline(VaR_K / kd, color='k', ls='--', lw=1.2,
              label='VaR$_K$, sqrt rule (%.0f)' % VaR_K)
ax[0].axvline(reg_cap / kd, color='k', ls=':', lw=1.4,
```

```
                label='3 x VaR$_K$ (%.0f)' % reg_cap)
ax[0].axvline(VaR_shock / kd, color='C3', ls='--', lw=1.2,
                label='shocked VaR (%.0f)' % VaR_shock)
ax[0].set_xlabel('%d-day linearized loss ($ thousand)' % K)
ax[0].set_ylabel('density')
ax[0].legend(fontsize=8)

days = np.arange(1, K + 1)
for cp, col, lab in [(corr_base, 'C0', 'no shock'), (corr_shock, 'C3', 'shoc
    lo, med, hi = np.quantile(cp, [0.05, 0.5, 0.95], axis=0)
    ax[1].fill_between(days, lo, hi, color=col, alpha=0.2)
    ax[1].plot(days, med, color=col, lw=1.6, marker='o', ms=3,
               label=lab + ' (median, 5-95%)')
ax[1].set_xlabel('day k')
ax[1].set_ylabel('EWMA correlation used on day k')
ax[1].set_xticks(days)
ax[1].legend(fontsize=8)

fig.tight_layout()
plt.show()
```

